

LaneLink

Author: Stolniceanu Iustin-Pavel

LaneLink is a Domain-Specific Language (DSL) designed to describe traffic intersection layouts, signal connections, and timing phases. It serves as a pedagogical example for building a compiler front-end using **Lex (Flex)** and **Yacc (Bison)**.

Features

- **Geometric Description:** Define lanes with IDs, directions (INCOMING/OUTGOING), and bearings (0-359 degrees).
- **Connectivity:** Map traffic flow between lanes with named connections.
- **Signal Phases:** Group connections into signal phases with specific durations.
- **Conflict Detection:** Automatically calculates geometric conflicts between crossing traffic paths.
- **Visualization:** Generates a JSON representation that can be visualized using a provided Python script.

Project Structure

```
.
├── src/
│   ├── lexer.l          # Lexical analyzer (Flex)
│   ├── parser.y          # Syntax analyzer (Bison/Yacc)
│   ├── IntersectionGraph.c # Core logic & Data structures
│   └── main.c            # Application entry point
├── include/
│   └── IntersectionGraph.h # Header definitions
├── examples/
│   ├── DSL_Prototype.txt  # Simple example
│   └── Complex_Intersection.txt # Advanced example
├── scripts/
│   └── visualize_intersection.py # Matplotlib-based visualizer
└── Makefile                # Build system
```

Prerequisites

To build and run LaneLink, you need: - gcc - lex - yacc - python3 (for visualization) - pip install matplotlib numpy

Build Instructions

Compile the project using the provided Makefile:

`make`

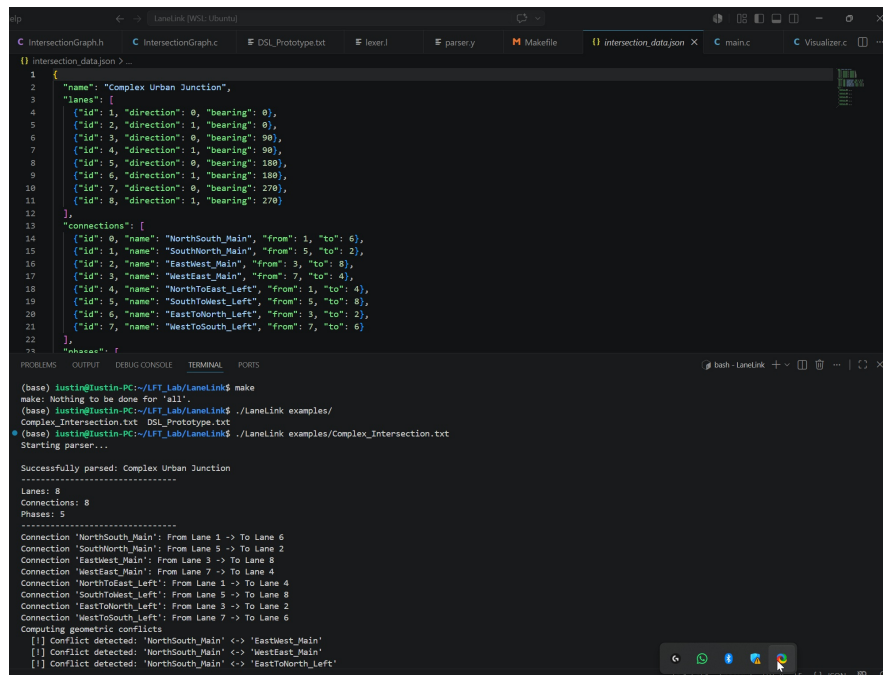
This will generate the LaneLink executable.

Usage

1. Parse a DSL file

Run the parser on an input file to generate the intersection data and compute conflicts:

```
./LaneLink examples/DSL_Prototype.txt
```



```
1 {
2   "name": "Complex Urban Junction",
3   "lanes": [
4     {"id": 1, "direction": 0, "bearing": 0},
5     {"id": 2, "direction": 1, "bearing": 0},
6     {"id": 3, "direction": 0, "bearing": 90},
7     {"id": 4, "direction": 1, "bearing": 90},
8     {"id": 5, "direction": 0, "bearing": 180},
9     {"id": 6, "direction": 1, "bearing": 180},
10    {"id": 7, "direction": 0, "bearing": 270},
11    {"id": 8, "direction": 1, "bearing": 270}
12  ],
13  "connections": [
14    {"id": 0, "name": "NorthSouth_Main", "from": 1, "to": 6},
15    {"id": 1, "name": "SouthNorth_Main", "from": 5, "to": 2},
16    {"id": 2, "name": "EastWest_Main", "from": 3, "to": 8},
17    {"id": 3, "name": "WestEast_Main", "from": 7, "to": 4},
18    {"id": 4, "name": "NorthToEast_Left", "from": 1, "to": 4},
19    {"id": 5, "name": "SouthToWest_Left", "from": 5, "to": 8},
20    {"id": 6, "name": "EastToNorth_Left", "from": 3, "to": 2},
21    {"id": 7, "name": "WestToSouth_Left", "from": 7, "to": 6}
22  ],
23  "phases": [
24  ]
25 }
```

```
(base) iustin@iustin-PC:~/LFT_Lab/LaneLink$ make
make: Nothing to be done for 'all'.
(base) iustin@iustin-PC:~/LFT_Lab/LaneLink$ ./LaneLink examples/
Complex_Intersection.txt DSL_Prototype.txt
(base) iustin@iustin-PC:~/LFT_Lab/LaneLink$ ./LaneLink examples/Complex_Intersection.txt
Starting parser...

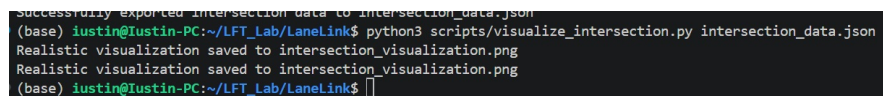
Successfully parsed: Complex Urban Junction
-----
Lanes: 8
Connections: 8
Phases: 5
-----
Connection 'NorthSouth_Main': From Lane 1 -> To Lane 6
Connection 'SouthNorth_Main': From Lane 5 -> To Lane 2
Connection 'EastWest_Main': From Lane 3 -> To Lane 8
Connection 'WestEast_Main': From Lane 7 -> To Lane 4
Connection 'NorthToEast_Left': From Lane 1 -> To Lane 4
Connection 'SouthToWest_Left': From Lane 5 -> To Lane 8
Connection 'EastToNorth_Left': From Lane 3 -> To Lane 2
Connection 'WestToSouth_Left': From Lane 7 -> To Lane 6
Computing geometric conflicts
[!] Conflict detected: 'NorthSouth_Main' <-> 'EastWest_Main'
[!] Conflict detected: 'NorthSouth_Main' <-> 'EastToNorth_Left'
```

This will output `intersection_data.json`.

2. Visualize the result

Use the Python script to generate a PNG visualization of the intersection:

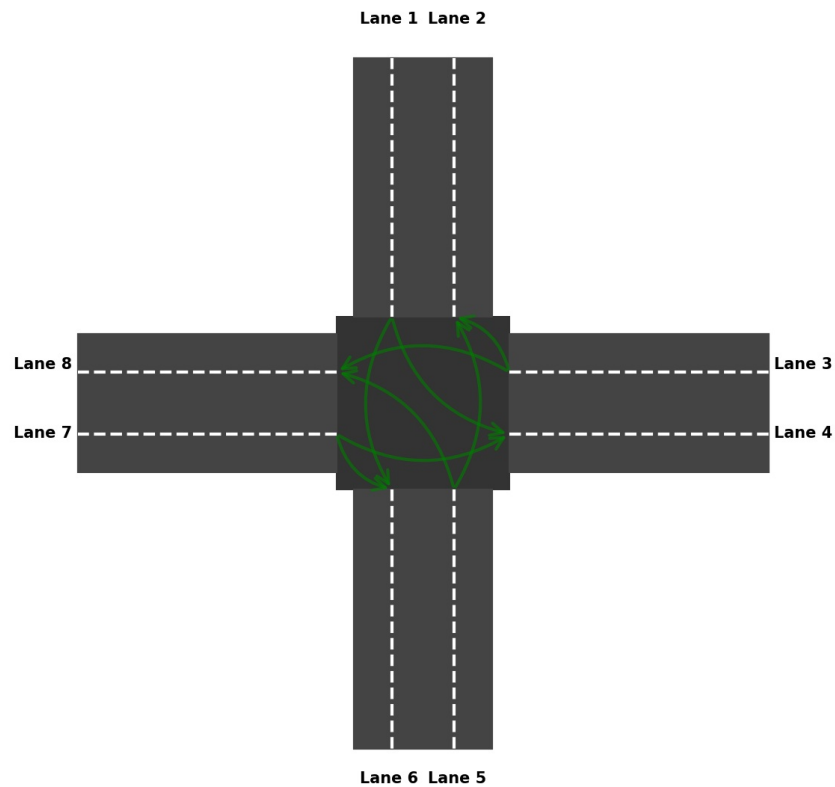
```
python3 scripts/visualize_intersection.py intersection_data.json
```



```
Successfully exported intersection data to intersection_data.json
(base) iustin@iustin-PC:~/LFT_Lab/LaneLink$ python3 scripts/visualize_intersection.py intersection_data.json
Realistic visualization saved to intersection_visualization.png
Realistic visualization saved to intersection_visualization.png
(base) iustin@iustin-PC:~/LFT_Lab/LaneLink$
```

The result will be saved as `intersection_visualization.png`. Example of an intersection generated by the program and visualized by the script:

Intersection Layout: Complex Urban Junction



DSL Syntax

An intersection is defined within an intersection block:

```
intersection "My Crossroads" {  
  // lane <ID> <DIRECTION> <BEARING>;  
  lane 1 INCOMING 0;  
  lane 2 OUTGOING 180;  
  
  // connect <FROM_ID> -> <TO_ID> as "ConnectionName";  
  connect 1 -> 2 as "NorthToSouth";  
  
  // phase { "ConnectionName1", ... } duration <MS>;  
  phase { "NorthToSouth" } duration 5000;  
}
```

- **Bearings:** 0 is North, 90 is East, 180 is South, 270 is West.
- **Directions:** INCOMING for traffic entering the intersection, OUTGOING for traffic leaving it.